

Anwendungssysteme

Anne

TODO

Reflection

Meaven

Testing

Aufgabe zu Reflection

-> öffnet Vorlesungsfolien 01 S. 26 - 28

```
public class User extends UserMutter { no usages new *
```

```
    public String name;
```

```
    private int age; 2 usages
```

```
> public int getAge() { return age; }
```

```
    public User(){ no usages new *
```

```
    }
```

```
    public User(String name, int age){ no usages new *
```

```
        this.name = name;
```

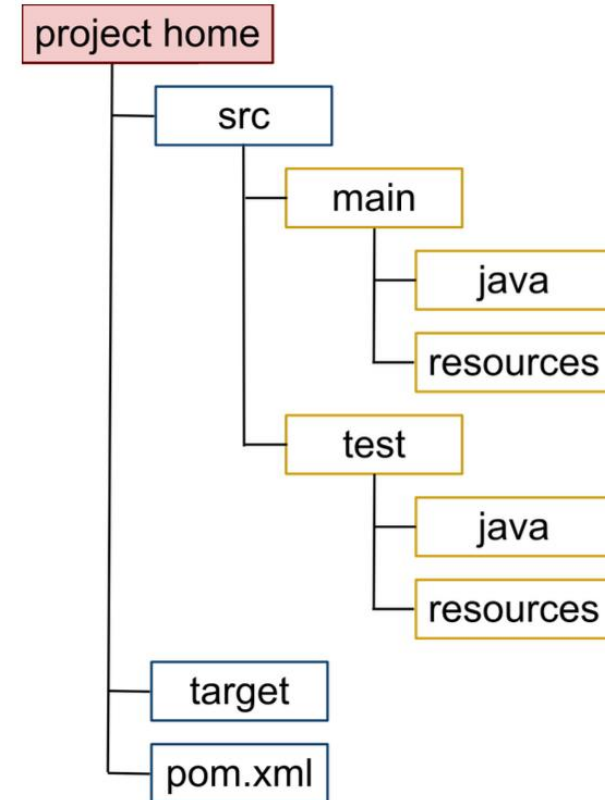
```
        this.age = age;
```

```
    }
```

```
}
```

Maven

- Build tool
- Verwaltet Abhängigkeiten
- Erzwingt Projektstruktur
- Ohne Maven kein einheitlichen Build
- Lädt Dependencies automatisch



https://upload.wikimedia.org/wikipedia/commons/c/cf/Maven_CoC.svg

POM.xml (Herzstück)

- `<groupId>com.example</groupId>`

-> (zu Welcher Organisation gehört Projekt, z.b org.springframework)

`<artifactId>mein-projekt</artifactId>` (Name deines Projektes) `<version>1.0.0</version>`

Typische Schreibweise:

1.0.0 → erste stabile Version

1.0.1 → Bugfix

2.0.0 → größere Änderungen

Manchmal auch:

1.0.0-SNAPSHOT → noch in Entwicklung

Beispiel

- Ich veröffentliche ein project -> dann kann jemand anderes es so nutzen:

```
<dependency>
```

```
  <groupId>com.example</groupId>
```

```
  <artifactId>mein-projekt</artifactId>
```

```
  <version>1.0.0</version>
```

```
</dependency>
```

Lombok in Maven

<dependency>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

<version>1.18.30</version>

<scope>provided</scope>

(Wird zur Kompilierung benötigt)

</dependency>

```
import lombok.Getter;  
import lombok.Setter;
```

@Getter

@Setter

```
public class User {  
    private String name;  
}
```

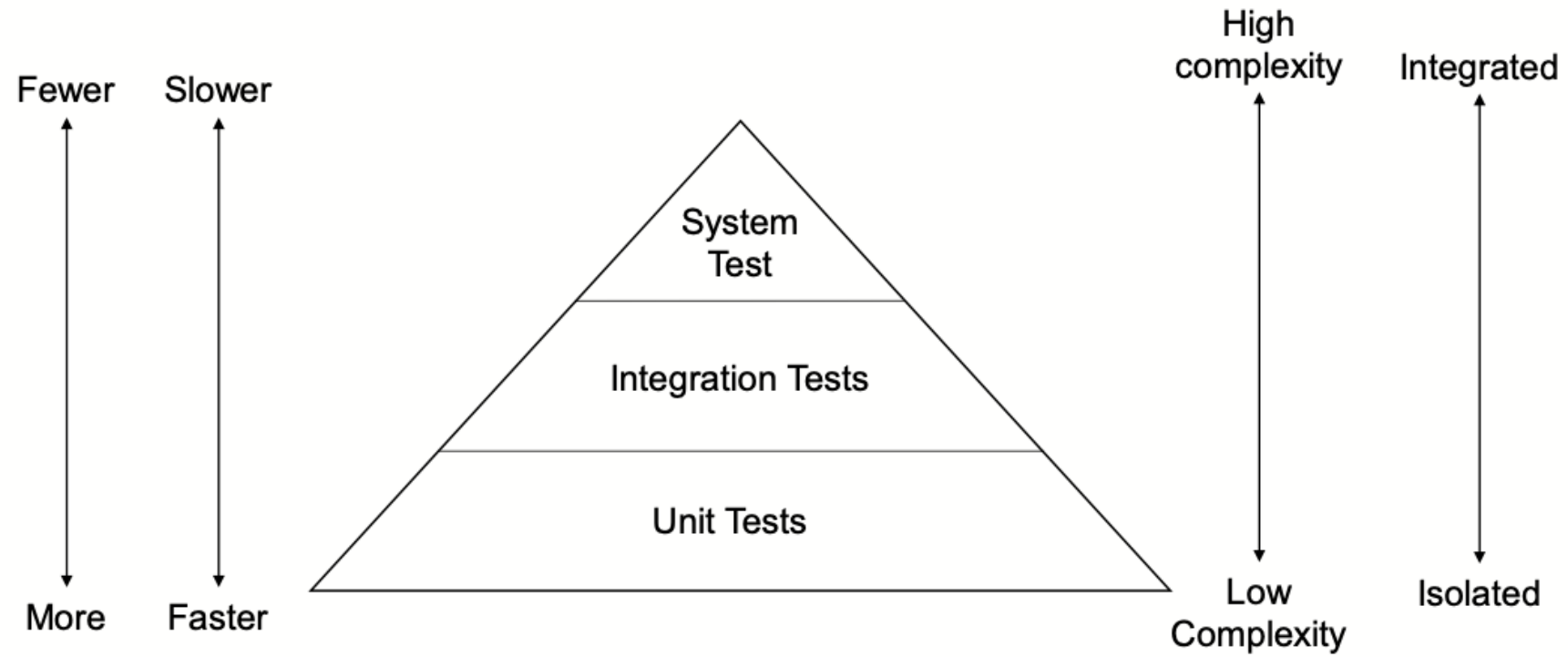

Aufgaben

- Erstellt jetzt ein neues Projekt in IntelliJ. Beim Erstellen gibt es die Option 'Maven' als Build System – wählt das aus
- Öffnen Sie das Maven-Repository. Dort finden wir zwei Libraries, die wir jetzt in unserem Code verwenden wollen: [Lombok](#) und [JUnit](#).
- Maven Repository (mvnrepository.com)
- Erstelle eine Klasse Haus in der du Lombok benutzt und damit automatisch eine toString Methode, einen Konstruktor und Getter und Setter erstellst

Testing

- **Unit-Tests:** untersuchen einen bestimmten Teil des Codes, z. B. eine Methode oder Klasse
- **Integrationstests:** untersuchen das Zusammenspiel mehrerer Komponenten des Systems
- **Systemtests:** untersuchen das gesamte System

The testing pyramid



Junit good to know

- **@DisplayName:** ändert den angezeigten Namen eines Tests (Standard ist der Methodename)
- **@BeforeAll:** führt eine Methode **einmal vor allen Tests** in dieser Klasse aus (muss zur Klasse gehören, also static sein)
- **@BeforeEach:** führt eine Methode **vor jedem einzelnen Test** aus
- **@Order(int num):** legt fest, in welcher Reihenfolge die Tests ausgeführt
@TestMethodOrder(MethodOrderer.OrderAnnotation.class) das muss dann über der Klasse stehen
- `assertEquals(expected, Methode)`
- `assertTrue(value)`
- `assertFalse(value)`
- `assertNull(value)`

Example: JUnit

```
public class CalculatorTest {
```

```
    private Calculator calc;
```

```
    @BeforeEach
```

```
    void init() {
```

```
        calc = new CalculatorImpl();
```

```
    }
```

```
    @Test
```

```
    @DisplayName("1+2=3")
```

```
    void testAdd() {
```

```
        assertEquals(3, calc.add(1, 2));
```

```
    }
```

GitHUb –
Code kommt
später

